TKDA: An Improved Method for K-degree Anonymity in Social Graphs

1st Nan Xiang
Wireless Networking and Mobile Computing Laboratory
Inner Mongolia University
Hohhot, China
nancyy6735@gmail.com

2nd Xuebin Ma
Wireless Networking and Mobile Computing Laboratory
Inner Mongolia University
Hohhot, China
csmaxuebin@imu.edu.cn

***Abstract*—Data anonymization is one of the most important directions in privacy-preserving. However, research shows that simple anonymization of data does not protect privacy. To solve this problem, we present a novel and effective algorithm named tree-based K-degree anonymity (TKDA). We devise a new anonymity sequence generation method to reduce the information loss for social graphs. Then, the dynamic anonymization process is implemented by a depth-first search (DFS) traversal algorithm. Finally, the graph modification algorithm based on the anonymous sequence can keep the original graph structure stable. Average Path Length (APL), Average Clustering Coefficient (ACC), and Transitivity (T) are employed to evaluate the method. Experimental results on several datasets show that TKDA is closer to the values of the original graphs on the correlated three experimental metrics, which indicates that TKDA portrays the real data in more detail and improves the utility of the released data.**

I. Introduction

Nowadays, with the rapid development of social networks (e.g., Twitter, Facebook, and TikTok), these social network service providers can collect and maintain data. More and more researchers have found that these data can fully utilize their potential social and economic benefits through in-depth mining and analysis, such as user behavior, community growth, disease transmission [1]. Meanwhile, the information can also be used to improve the quality of user services (e.g., smart cities, advertising). Existing vast sensitive users' information in these social networks, it is very likely to disclose user privacy if the data is directly released. Thus, it becomes a challenging problem to strike a balance between users' privacy and data utility. K-anonymity technique is considered as one of the most popular approaches to solve this problem[2]. The K-anonymity tries to prevent the attacker from recognizing every user from other K-1 users and knowing the users' identity. K-anonymity can also prevent background knowledge-based attacks [3].

Generally speaking, there are two types of social networks data: relational data and graph data [4]. Anonymizing characteristics of relational data is usually easy and can be done by randomization, generalization, suppression. However, anonymization of graph data is more difficult than anonymization of relational data [5]. Since it will affect the whole structure of the graph if one node changes. Some existing mechanisms are mainly used to solve the privacy issues of relational data publishing, such as l-diversity [6], t-closeness [7] and differential privacy [8]. Most existing methods cannot be directly applied to social networks' data release. The main solution for graph data publishing in social networks is the K-degree anonymity model. The K-degree anonymity model [9] is a robust privacy-preserving technique after the K-anonymity mechanism. K-degree anonymity can guarantee that every node has the same degree with at least K-1 other nodes in the graph for preserving privacy.

This paper proposes a K-degree anonymity based on tree structure privacy protection algorithm. Depth-first search traversal algorithm and binary tree structure ensure the anonymized degree sequence. It is essential to protect the privacy of users' sensitive information before data release [10]. The work of this paper is summarized as follows:

- We propose a new K-degree anonymization algorithm based on tree structures for undirected graphs to further improve data utility while guaranteeing users' data privacy.
- We devise a new anonymity sequence generation algorithm by the DFS traversal method, which creates the K-degree anonymity sequence. This method achieves a dynamic anonymization process that can retain more edges of the original graph. In addition, it can decrease the differences between the anonymized graph and the original graph.
- Experiments focus on three real social network datasets. And the results show that TKDA significantly demonstrates the metrics of real data and improves the utility of published data under the same privacy level.

The rest of this paper is organized as follows. Section II reviews related work on graph data anonymization. Section III introduces some relevant basics and the definition of K-degree anonymity graphs. Section IV details the process of TKDA. Section V analyzes and summarizes the experimental results. Finally, the conclusions are presented in Section VI.

II. RELATED WORK

As many web datasets get access to the public, the problem of users' privacy becomes a hot topic. Among them, data anonymization has been one of the researchers' focuses on privacy protection. In order to protect data privacy, There are

978-1-6654-9792-3/22/$31.00 copyright2022 IEEE

two kinds of K-degree anonymization techniques, broadly categorized as clustering-based approaches and graph modification approaches. The clustering-based K-anonymity technique is not applied to large networks since the time cost of the approach is usually high. Therefore, we focus on the field of graph modification. Next, we introduce the graph modification method.

There are two kinds of graph modification techniques, the first one is to modify only edges, not to modify nodes. Liu and Terzi [9] first introduced the concept of K-degree anonymity, which means that the graph is a K-degree anonymous graph if each node in the graph has at least K-1 other nodes with the same degree. The algorithm can be applied to dynamic network datasets. This method can reduce the utility of the graph, and the algorithm does not apply to large networks. To decrease the errors caused by excessive modification of edges and nodes during graph anonymization, Lu et al. [14] proposed a fast greedy algorithm named FKDA, which anonymized social networks by adding edges and making the anonymization degree sequence simultaneously. The method can improve the utility of the graph, but the runtime increases with increasing the value of K. Jordi et al.[15] proposed the UMGA algorithm, which used univariate microaggregation to generate anonymized graphs and selected edges by calculating the neighborhood centrality score of each edge. Although the reduced information loss makes the graph more practical, the running time increases as the value of K increases.

The second method is to modify simultaneously edges and nodes. Chester et al. [16, 17] proposed an algorithm for adding virtual nodes, which achieves K-degree anonymity by adding new nodes and connecting them. Although the number of nodes and edges added by this algorithm is minimal, the original graph structure is still not well preserved. Ma [18] in 2015 proposed a K-degree graph anonymization algorithm (KDVEM) by modifying edges to achieve anonymized graphs, which presented a two-step algorithm to anonymize the degree sequence of the graph. Firstly, the algorithm found a target degree for each node in the graph. And then, some edges are added between the vertices to increase the degree of the vertices. If there exist vertices that are still not anonymous, Chester's [17] algorithm will be used. This algorithm can achieve the anonymity requirement by adding the minimum number of new vertices and edges. This is why the method can retain graphs' features.

Compared with other anonymization techniques, graph modification methods are simple and easy to implement. In summary, graph modification can retain more original graph characteristics while satisfying the desired privacy requirements. Therefore, we use the graph modification technique in this paper.

III. PRELIMINARIES

We consider the undirected graph $G = (V, E)$ without labels and weights, where V denotes the set of nodes and E is the set of edges. Each node represents an individual in the social network, and each edge means a connection between two users. More precise definitions of K-anonymity and K-degree anonymity are as follows.

Definition 1 (K-anonymity [9]). *Suppose that given each vector V_i in V appears at least K times, then we can say that a vector of integers V is K-anonymity.*

Definition 2 (K-degree anonymous graph [9]). *Give an undirected graph $G = (V, E)$, graph G is a K-degree anonymous graph if its degree sequence d satisfies K- anonymity.*

Definition 3 (Information loss function [10]). *Using the function $dist$ calculated as the distance between graph G and the anonymous graph $\tilde{G}$, computed as Equation (1).*

$$dis(G, \tilde{G}) = \sum_{i=1}^{n} |d_i - \tilde{d}_i| \tag{1}$$

where the lower the value of dis, the lower the information loss of anonymized graph and the higher the data utility.

Theorem 1 (Havel-Hakimi theorem [19]). Let $d = (s, t_1, ..., t_s, d_1, ..., d_n)$ be a finite list of nonnegative integers that is nonincreasing. $d^{'} = (t_1 - 1, ..., t_s - 1, d_1, ..., d_n)$ be a second finite list of nonnegative integers that is rearranged to be nonincreasing. List d is graphic if and only if list $d^{'}$ is graphic.

IV. PROBLEM SOLUTION

To protect users' privacy, we present a new approach called TKDA (Tree K-Degree Anonymity), which is designed to achieve K-degree anonymity on undirected and unlabelled social networks. The privacy of individuals can be protected through the results since the adversary cannot infer the identity of any person for K-degree anonymous graphs. The specific process and implementation of TKDA are described in this section.

A. Algorithm Description

The algorithm is based on a two-step approach:

(1) Degree sequence anonymization: The original degree sequence d is generated from the original graph G, then we construct the K-degree anonymity sequence based on the given K values and degree sequence d. Using a binary tree structure compresses the graph. Firstly, the tree structure is built based on the degree sequence. Secondly, the target node degree is assigned for each group in the partition. Finally, the degree sequence is grouped according to the K value to generate an anonymous degree sequence.

(2) Graph modification: We construct the anonymous graph based on the anonymous degree sequence obtained in Section IV.B by using the three basic edge modification operations. K-degree anonymity protects users' privacy by modifying edges to generate a new anonymous graph $\tilde{G} = (V, E)$, where $\tilde{G}$ represents the anonymous graph. The specific process is shown in Algorithm 1.

Algorithm 1 TKDA

Input: The original graph G, Anonymous parameter K.
Output: The anonymous graph $\tilde{G}$.

1: d = the degree sequence of the original graph G
2: T = ConstructPartitionTree(G)
3: anonymous sequence($\tilde{d}$) = Seq_Partition(T)
4: Flag = Havel-Hakimi($\tilde{d}$)
5: **if** Flag == False **then**
6: $\tilde{d}$=$\tilde{d}$+random_noise
7: **else**
8: continue
9: **end if**
10: anonymous($\tilde{G}$) = GraphConstruct($\tilde{d}$, G)
11: **return** anonymous($\tilde{G}$)

Algorithm 1 describes the detailed process of TKDA. The original degree sequence is obtained from the original graph in line 1. The algorithm for grouping the tree from line 2 is shown in Section IV.B. The anonymity sequence is created by DFS according to line 3. The Havel-Hakimi theorem (Theorem 1) decides whether the degree sequence can construct a graph from line 4. If Flag is false, adding random noise sets the value as 1. Otherwise, the rest of the algorithm is continued. The GraphConstruct algorithm of line 10 presents delete, add, switch edges, and section IV.C introduces the specific operations. In the end, it returns the anonymous graph by graph construction algorithm.

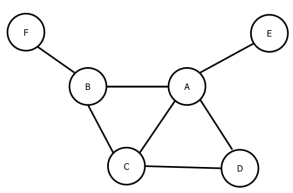


Fig. 1. Original graph.

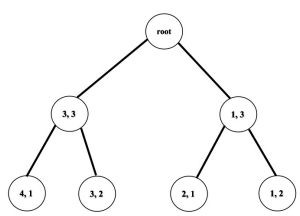


Fig. 2. Partition tree.

B. Anonymous degree sequence

The goal of the first step is to anonymize the degree sequence of the original network. We use the idea of Huffman tree generation to achieve the grouping tree and compute these values for each group that minimize the distance from the original degree sequence d. The algorithm of this part will be explained in detail.

1) Construct degree sequence Tree: In the process of producing an anonymous degree sequence, the binary tree is introduced to compress the degree sequence of the graph for improving space efficiency. In this binary tree, the leaf nodes are all composed of a pair of (D, DC), where D denotes the degree of the node and DC denotes the number of the node's degree. The parent nodes in the tree are aggregated from the children nodes, which contain the degree and the count of degree. This is why the count of nodes in this tree is much less than the social graph nodes. In order to build the tree, the nodes are sorted in descending order according to their degree value. For example, the degree sequence in Figure 1 is [4, 3, 3, 2, 1, 1]. So, the aggregate vector of this graph is [(4, 1), (3, 2), (2, 1), (1, 2)]. (4, 1) represents only one node with a degree of 4. The aggregate vector is defined as follows.

$$a_v(G) = \{(D, DC) | D \in degree_seq(G), DC = count(D_i)\} \tag{2}$$

Using the idea of bottom-up construction build the tree. A two-dimensional list will be generated from the degree sequence after initializing the leaf nodes with aggregate vectors. The aggregated new vector is calculated as follows.

$$(D_i, DC_i) \cup (D_j, DC_j) = (TD(v), TDC(v)) \tag{3}$$

$$TDC(v) = DC_i + DC_j \tag{4}$$

$$TD(v) = \begin{cases} \lceil \frac{((D_i * DC_i) + (D_j * DC_j))}{TDC(v)} \rceil, DC_i \geq DC_j \\ \lfloor \frac{((D_i * DC_i) + (D_j * DC_j))}{TDC(v)} \rfloor, DC_i < DC_j \end{cases} \tag{5}$$

where v_i and v_j are the two nodes to be merged. v is the new target node. D_i denotes the degree of node v_i. DC_i represents the count of degree D_i in the degree sequence of the graph. $TD(v)$ denotes the target degree of the new node. If $DC_i \geq DC_j$, $TD(v)$ is the sum of $(D_i{*}DC_i)$ and $(D_j{*}DC_j)$ divided by the count of the new node' target degree. And the result obtained by rounding ceil is the degree of the new target node. Otherwise, the degree of the new node is round floor the result. $TDC(v)$ is the count of the new node' target degree. The resulting partition tree is shown in Figure 2.

Algorithm 2 ConstructPartitionTree

Input: Original graph G.
Output: The tree structure of graph G.

Generate a degree sequence d from the original graph G
Create aggregate vector (D, DC) of the degree sequence d
Initiate the leaves of the tree by aggregate vectors (D, DC)
Queue Q is sorted by degree value of the degree sequence d in decreasing order
while len(Q) > 0 **do**
E $\leftarrow$ the first elements in queue Q
Merge the node (D_i, DC_i) and node (D_j, DC_j) $\rightarrow$ new node (TD, TDC) in E
Q $\leftarrow$ Q-{E}
end while
return The tree structure of graph G

To construct a partition tree by Algorithm 2, we calculate the degree and the count of all nodes' degrees in the graph $G = (V, E)$. The two nodes with the largest degree in degree sequence are to merge a new node after sorting in descending order according to the degree. We add the newly generated nodes to the list and remove these two nodes from the list. Repeat the above steps until the list has only one node, which is the tree's root.

Algorithm 3 Seq_Partition

Input: The tree structure of graph G, privacy level K, two-dimensional list m.

Output: Anonymous sequence $\tilde{d}$.

```
T ← The tree structure of graph G
myStack.append(T.root)
while myStack do
    now_node = myStack.pop()
    if now_node.DC > K then
        if now_node.left.DC < K or now_node.right.DC < K then
            m.append([now_node.D, now_node.DC])
        else
            myStack(now_node.left)
            myStack(now_node.right)
        end if
    end if
end while
d̃ ← m
return d̃
```

2) Degree sequence partition: To create the anonymous degree sequence, our algorithm uses the depth-first search (DFS) approach to traversal the tree. In Algorithm 3, the partition tree is traversed starting from the root of the tree. If the count of the node's degree is more than K value, we move to its children subtree and continue the loop until the count of degree of the children is less than the value of K. The traverse algorithm goes back to the parent node and marks the node's parent node. And we continue to traverse the adjacent node. Finally, these marked nodes are taken as output. As shown in Figure 3, the partition is decided by the DFS traversal algorithm. The shaded nodes {(3, 3), (1, 3)} are reached according to the K value. Thus, the anonymous degree sequence of the graph is {(3, 3, 3), (1, 1, 1)}.

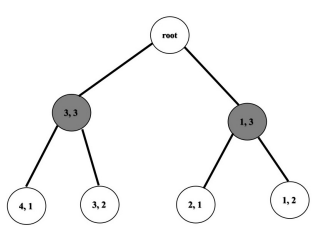


Fig. 3. Decide partition.

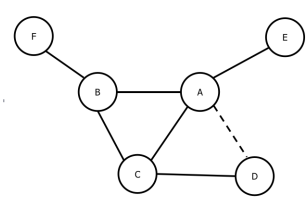


Fig. 4. Anonymous graph.

C. Graph modification

We make the anonymous graphs based on the anonymity degree sequence in this section. The original network needs to be changed to make it satisfy K-degree anonymity in the mechanism of TKDA. The social network satisfies K-degree anonymity if the degree sequence of the social network is K-anonymity according to Definition 2. Therefore, the anonymity sequence $\tilde{d}$ is changed from the original degree sequence d. In the process of graph modification, the set D is the anonymous degree sequence $\tilde{d}$ minus the original degree sequence d. $d_v^{'} \in D$ denotes the required degree of the node v. The set $d_v^{'} > 0$ means that the node needs to increase edges. $d_v^{'} < 0$ denotes the node needs to decrease edges. $d_v^{'}$ = 0 shows the node satisfies K-anonymity and does not need to be changed. Suppose that all $d_v^{'} > 0$ means nodes with d $>$ 0 form the set V^+ and all $d_v^{'} < 0$ nodes with d $<$ 0 form the set V^-. Given a set E of social network edges, we used three basic operations [15].

Add edge: If v_i and v_j are both nodes in V^+ and there is no edge of (v_i, v_j), we can add an edge between them. Then the degree of v_i and v_j are added by one.

Switch edge: If the node v_i in V^- and the node v_j in V^+, it means that v_i needs to delete edges and v_j needs to add edges. If v_i is connected to v_b and v_j is not connected to v_b, the edge of (v_i, v_b) is deleted and an edge of (v_j, v_b) is added. In this case, v_b remains the same degree, degree of the node v_i is reduced by one and degree of the node v_j increases by one.

Delete edge: If v_i and v_j in V^-, v_i and v_a are connected by an edge. There is one edge of (v_j, v_b), v_a and v_b do not need to change. Then we delete edges of (v_i, v_a) and (v_j, v_b). The edge of (v_a, v_b) is added. It ensures that v_a and v_b remain unchanged while the degree of v_i and v_j are reduced by one.

For example, the degree original sequence is $\{v_1, v_2, v_3, ..., v_6\} = \{4, 3, 3, 2, 1, 1\}$ in Figure 1. The anonymous degree sequence obtained according to Figure 2 is $\{v_1, v_2, v_3, ..., v_6\} = \{3, 3, 3, 1, 1, 1\}$. The partition tree is formed as Figure 3. So $V^- = \{v_1, v_4\}$ means the degree of node $v_1(A)$, $v_4(D)$ should be reduced by one. The edge connected to A and D is deleted, and the final anonymized graph is shown in Figure 4.

V. EXPERIMENTS ANALYSIS

A. Time complexity analysis

In the first step of the algorithm, the dataset is scanned once to build the aggregation vector of the graph, so the complexity of scanning the dataset is O(n), where n is the size of the dataset. Since the tree is built from the bottom up, similar to the process of building a heap, the complexity of building the tree structure is O(m), where m indicates the number of leaf nodes of the tree. Meanwhile, DFS traverses the tree to decide the grouping, since the height of the tree is O(m)∼O(logm), the time complexity of deciding the grouping is O(km)∼O(klogm), k indicates the number of branches that pass through the root during traversal. So the total time complexity of the first step is O(n+m+km)∼O(n+m+klogm). In the second step of the algorithm, the graph modification problem is a P-problem for random edge selection, but it can be resolved in polynomial time by computing the log_2 of all possible combinations.

B. Experimental evaluation metrics

In this part, we apply the proposed method on three datasets. Table 1 shows the features of these datasets. The first of these datasets is Polbooks, which is a network of books

TABLE I
EXPERIMENTAL DATASETS

Datasets	Edges	Nodes	ACC	T	APL
Polbooks	441	105	0.487	0.348	0.308
Ca-AstroPh	198110	18772	0.677	0.318	4.194
Ca-GrQc	14496	5242	0.530	0.630	6.049

about US politics. The following two datasets are real social networks from the e-print arXiv. These datasets are available on the Stanford University website [20]. There are some crucial measures to evaluate the effectiveness of the proposed method, such as ACC (Average Clustering Coefficient), T (Transitivity), APL (Average Path Length). The utility of the graph is better if the values of the anonymous graph are closer to the values of the original graph.

Average clustering coefficient: ACC is an indicator of the graph, which reflects the average number of neighbors of every vertex. The clustering coefficient of a vertex is the ratio of possible triangles through the vertex. The average clustering coefficient of a graph is the average of clustering coefficient for all vertices. The definition is as follows.

$$ACC(G) = \sum_{u=1}^{n} \frac{2T(u)}{deg(u)(deg(u)-1)} \tag{6}$$

where $T(u)$ is the number of triangles passing through node u and $deg(u)$ is the degree of u.

Transitivity: To calculate the Transitivity of a graph is actually to calculate the ratio of the triangles in the whole graph and the three connected triangles, which reflects the proportion of the triangles in the graph, as defined below.

$$T = \frac{3 * \triangle}{\wedge} \tag{7}$$

where $\triangle$ is the number of triangles and $\wedge$ is the number of connected triangles.

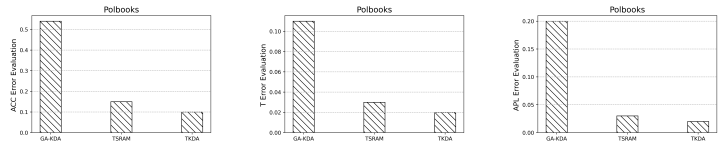
Average Path Length: APL is the last metric of a graph, which measures the average of the shortest paths between nodes in the graph. The social distance between all connectable pairs of a graph is measured by the average path length (APL). It is defined as follows.

$$APL(G) = \frac{\sum_{(u,v)\in Pair} SPL(u,v)}{|Pair|} \tag{8}$$

where $Pair$ denotes all reachable vertex pairs and $SPL(u,v)$ denotes the shortest path length between vertices u and v. It is a measure of the efficiency of information or mass transport on the network.

C. Analysis of experimental results

In order to evaluate the effectiveness of TKDA, six algorithms that aim at retaining the characteristics of the graph are used. GA-KDA [4] is a combination of community detection and genetic algorithm to generate an anonymous graph. TSRAM [11] uses a first-time tree structure to generate an anonymous graph, which has the advantage of saving more time cost. Priority [9], FKDA [14], Vertexadd [16], KDVEM [18] four algorithms are more classical algorithms, so we choose to use them for experimental comparison. They are not introduced here since these four methods have been described in detail in Section IV. For each dataset and algorithm, we vary K from 2 to 50 and compare the results obtained on T, ACC, APL. The experimental results are shown in these figures.



(a) ACC Error Evaluation (b) T Error Evaluation (c) APL Error Evaluation

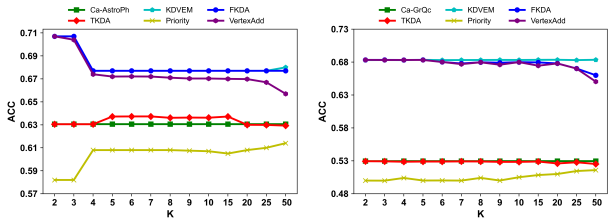
Fig. 5. Error Evaluation on Polbooks.

In Figure 5, we compare TKDA with GA-KDA and TSRAM algorithms, both of which are the latest algorithms. Polbooks is a small undirected network dataset. Error evaluation is computed as [11]:

$$EV = \frac{1}{n}\sum_{i=2}^{n} |a_i - a|$$

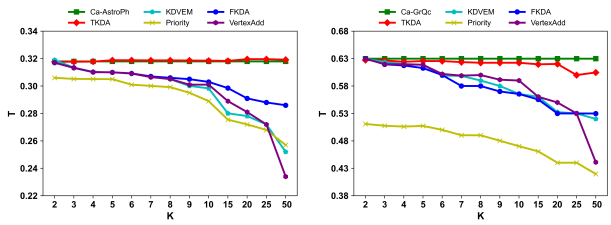
where n is the number of K values, a_i stands for a graph attribute for a value of K, and a is the value for the original graph.

Comparing the three methods in three metrics separately in Figure 5, we can find that TKDA has the best performance. Taking Figure 5 (a) as an example, the error evaluation of the average clustering coefficient of GA-KDA reaches 0.5, while the value of TSRAM varies in the range between 0.1 and 0.2. Thus the error evaluation of TKDA is smaller than the first two methods. As can be seen from Figure 6, the data utility of TKDA is higher when the privacy parameter K value is the same.



(a) Ca-AstroPh (b) Ca-GrQc

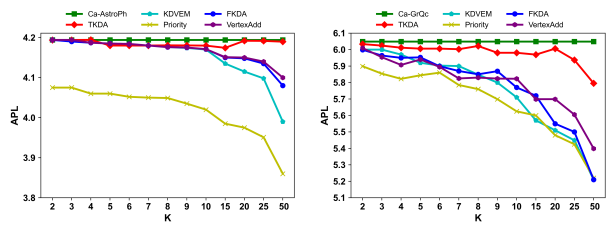
Fig. 6. Average clustering coefficient.



(a) Ca-AstroPh (b) Ca-GrQc

Fig. 7. Transitivity.

We report the values of the average clustering coefficient of anonymous graphs using two datasets in Figure 6. The green



(a) Ca-AstroPh (b) Ca-GrQc

Fig. 8. Average shortest path length.

line represents the value of the average clustering coefficient for the original graph. Figure 6 (a) shows that TKDA is slightly larger than the original value ($K \in [5, 15]$), the anonymization process increases the average clustering coefficient of the output graph since new connections are added. The ACC value of TKDA is the closest to the original data from Figure 6 (b). In Figure 6, TKDA has tiny changes of average clustering coefficient caused by the edge operation when implementing different K-value anonymized networks. TKDA is closest to the ACC of the original dataset.

Figure 7 shows the effect of K on the transitivity of the five algorithms over two real datasets. The difference between the transitivity of TKDA and the value of the original data set is close to zero on Ca-AstroPh. The transitivity index starts to decrease on Ca-GrQc when K=25. This is because the operation of switch edges changes the structure of the graph to different extents, which leads to the change of the T value. In Figure 7, the transitivity value of TKDA is basically the same as the value of the original network when the K value increases.

Figure 8 compares the average path length (APL) indicator of four algorithms after networks are anonymized. We can find that TKDA is almost the same as the values of the other three methods when $K \in [2, 15]$. However, the other three algorithms show a significant decrease in the average path length due to adding new edges to the original graph and making the vertices with the shortest path length smaller. TKDA is almost equal to the APL index of the original dataset in Figure 8 (a). TKDA starts to rise respectively at K = 8 and K = 20 in Figure 8 (b), which shows a decreasing trend as a whole. The reason for the decrease of ACC value is that TKDA deletes some edges of the graph when K = 20. As a result, we can find that TKDA performs better in the average path length metric on all datasets.

In conclusion, the evaluation results show that the ACC, APL, and T metrics of TKDA are closer to those of the original dataset. TKDA always has the best performance while ensuring the privacy of user data compared to other approaches.

VI. CONCLUSION

How to improve the utility of data while protecting privacy has become one of the significant issues to be solved urgently. To this end, this paper focuses on the anonymization of social graphs using K-degree Anonymity. To reduce the information loss of the graph and achieve a dynamic anonymization process, we propose a new K-degree anonymity based on tree structure algorithm. Collectively, experimental results show that TKDA guarantees user privacy while the anonymous graph is closer to the original graph and improves data utility.

ACKNOWLEDGMENT

This paper is supported by the Science and Technology Program of Inner Mongolia Autonomous Region (Grant No. 2019GG116).

REFERENCES

[1] Shibchurn J, Yan X. Information disclosure on social networking sites: An intrinsic-extrinsic motivation perspective[J]. Computers in Human Behavior, 2015, 44: 103-117.
[2] El Emam K, Dankar F K. Protecting privacy using k-anonymity[J]. Journal of the American Medical Informatics Association, 2008, 15(5):627-637.
[3] Hao Y, Cao H, Hu C, et al. K-anonymity for social networks containing rich structural and textual information[J]. Social Network Analysis and Mining, 2014, 4(1): 223.
[4] Rajabzadeh S, Shahsafi P, Khoramnejadi M. A graph modification approach for k-anonymity in social networks using the genetic algorithm[J]. Social Network Analysis and Mining, 2020, 10: 1-17.
[5] Zhou B, Pei J. Preserving privacy in social networks against neighborhood attacks[C]//2008 IEEE 24th International Conference on Data Engineering. IEEE, 2008: 506-515.
[6] Machanavajjhala A, Kifer D, Gehrke J, et al. l-diversity: Privacy beyond k-anonymity[J]. ACM Transactions on Knowledge Discovery from Data (TKDD), 2007, 1(1): 3-es.
[7] Li N, Li T, Venkatasubramanian S. t-closeness: Privacy beyond k-anonymity and l-diversity[C]//2007 IEEE 23rd International Conference on Data Engineering. IEEE, 2007: 106-115.
[8] Dwork C. Differential privacy[C]//International Colloquium on Automata, Languages, and Programming. Springer, Berlin, Heidelberg, 2006: 1-12.
[9] Liu K, Terzi E. Towards identity anonymization on graphs[C]//Proceedings of the 2008 ACM SIGMOD international conference on Management of data. 2008: 93-106.
[10] Zhang X, Liu J, Li J, et al. Large-scale dynamic social network directed graph k-in&out-degree anonymity algorithm for protecting community structure[J]. IEEE Access, 2019, 7: 108371-108383.
[11] Kiabod M, Dehkordi M N, Barekatain B. TSRAM: A time-saving k-degree anonymization method in social network[J]. Expert Systems with Applications, 2019, 125: 378-396.
[12] Backstrom L, Dwork C, Kleinberg J. Wherefore art thou R3579X? Anonymized social networks, hidden patterns, and structural steganography[C]//. Proceedings of the 16th international conference on World Wide Web. 2007: 181-190.
[13] Zhou B, Pei J, Luk W S. A brief survey on anonymization techniques for privacy preserving publishing of social network data[J]. ACM Sigkdd Explorations Newsletter, 2008, 10(2): 12-22.
[14] Lu X, Song Y, Bressan S. Fast identity anonymization on graphs[C]//International Conference on Database and Expert Systems Applications. springer, Berlin, Heidelberg, 2012: 281-295.
[15] Casas-Roma J, Herrera-Joancomartí J, Torra V. k-Degree anonymity and edge selection: improving data utility in large networks[J]. Knowledge and Information Systems, 2017, 50(2): 447-474.
[16] Chester S, Kapron B M, Ramesh G, et al. k-Anonymization of Social Networks by Vertex Addition[J]. ADBIS (2), 2011, 789: 107-116.
[17] Chester S, Kapron B M, Ramesh G, et al. Why Waldo befriended the dummy? k-Anonymization of social networks with pseudo-nodes[J]. Social Network Analysis and Mining, 2013, 3(3): 381-399.
[18] Ma T, Zhang Y, Cao J, et al. KDVEM: A k-degree anonymity with vertex and edge modification algorithm[J]. Computing, 2015, 97(12): 1165-1184.
[19] Griggs J R, Kleitman D J. Independence and the Havel-Hakimi residue[J]. Discrete Mathematics, 1994, 127(1-3): 209-212.
[20] Stanford large network dataset collection. http://snap.stanford.edu/data/